

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Developing the System Requirements Specification for Your Own Project

Requirements and System Planning

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 5
Output	System Requirements Specification (SRS), requirements-to-feature traceability matrix, and online requirements review
Maximum Score	80 points

Student / Project Information

Block: <u>3 - A</u>	Date: October 4, 2026
System Title: SariwaBa?: Fish Freshness Classification System using Machine Learning and Mobile Computer Vision	Instructor: Jean Y. Villasin, MIT
Members: Marc Elson M. Rebato, Benedict A. Mendoza	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

1. Activity Alignment and Purpose

Topic Learning Outcome: Analyze the requirements of the group's semester web-system project and produce a scoped requirements specification aligned with the prescribed architecture and selected development methodology.

Course Outcome Alignment: Supports semester-project planning by translating the community problem, user needs, system boundaries, and project constraints into clear and testable requirements that can guide design,

implementation, testing, and review.

Syllabus Assessment: System Requirements Specification; requirements-to-feature matrix; online requirements review.

Purpose of the Activity

Activities 2, 3, and 4 established your project architecture, technology stack, and development methodology. In Activity 5, your team will convert the approved community problem and intended user needs into a controlled requirements baseline. You will define functional and non-functional requirements, user roles and permissions, system scope and boundaries, use cases and/or user stories, feature priorities, acceptance criteria, and project constraints. The resulting SRS and traceability matrix will become the reference for the succeeding design, implementation, database, API, testing, and review activities.

Important distinction: A requirement states what the system must do or what quality/constraint it must satisfy. A feature groups related capabilities that users can recognize. Scope defines what the project will and will not deliver. Architecture describes how the solution is organized. Do not replace a requirement with a technology name unless the technology itself is a prescribed project constraint.

2. Intended Learning Objectives

- Analyze the community or organizational problem, target users, stakeholders, and desired system outcomes.
- Write clear, atomic, and testable functional requirements using project-specific actions and data.
- Specify measurable non-functional requirements for security, performance, usability, reliability, compatibility, maintainability, and other relevant quality needs.
- Define user roles, permissions, data-access boundaries, and responsibilities using the principle of least privilege.
- Establish the system scope, out-of-scope items, assumptions, dependencies, and project constraints. • Refine use cases and/or user stories, feature priorities, business rules, and acceptance criteria based on the selected development methodology.
- Assemble a coherent SRS and requirements-to-feature traceability matrix aligned with Flutter Web, the REST API, PostgreSQL, and the team's selected methodology, then prepare the requirements baseline for online review.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to requirements analysis, refinement, traceability, and review.

Activity Item	Requirement
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled semester-project consultation and online requirements review.
Primary Output	One project-specific System Requirements Specification (SRS) and one Requirements-to-Feature Traceability Matrix.

Supporting Output	Problem/user analysis, functional and non-functional requirements worksheets, role-permission matrix, scope and boundary worksheet, use-case/user-story refinement, feature-priority and acceptance-criteria plan, and review notes.
Recommended Tools	Word/Google Docs, diagrams.net/Draw.io or equivalent for use-case diagrams, spreadsheet/table tool for traceability, Git/GitHub, and the team's current project board or documentation workspace.
Review Format	Online requirements review through the course platform, with instructor consultation or revision requests when requirements are incomplete, ambiguous, out of scope, or inconsistent with the prescribed architecture.

4. Pre-Laboratory Requirements

- Initial list of target users, stakeholders, major features, data to be collected, and any external services or integrations.
- Available requirement evidence such as stakeholder interview notes, observation results, existing forms/processes, instructor feedback, policy/rule references, or project consultations.
- Access to the project GitHub repository or shared documentation workspace where the SRS, diagrams, review notes, and traceability matrix will be maintained.

5. Part A - Community Problem and Target-User Analysis

Begin with the actual problem before writing features. Requirements should be traceable to a real user need, stakeholder rule, project constraint, or system-quality objective. Complete the project-context worksheet and the target-user/stakeholder analysis before defining detailed requirements.

Project Context Item	Your Project Evidence / Statement
Official system title	SariwaBa?: Fish Freshness Classification System using Machine Learning and Mobile Computer Vision
Community / organizational problem	Subjective, manual assessment of fish freshness at local public markets leads to inconsistent quality evaluation, consumer distrust, pricing disputes, and potential health hazards from spoiled seafood.
Current process or problem evidence	Sellers and buyers rely on subjective visual and sensory checks (e.g., eye clarity, gill color, firmness), leading to frequent pricing disputes during transactions.
Target community / organization	Local public fish markets, seafood vendors, and retail buyers.
Primary users / beneficiaries	Fish Vendors / Sellers, Retail Consumers / Buyers, and Unauthenticated Guests.
Secondary users / stakeholders	System/Backend Administrators (Django Admin level)
Desired system outcome and success indicator	Deliver an automated image-based freshness classification tool achieving a classification accuracy rate > 85% and an end-to-end classification response speed under 3 seconds per image scan.

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Fish Vendor/Seller	Build buyer trust, justify fair pricing, and verify inventory quality using standardized visual classification.	Signs in via Google OAuth 2.0, performs freshness scans on market inventory, views classification results, and manages vendor scan history logs.	Market observation of vendor-buyer price disputes caused by subjective visual grading.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

User / Stakeholder	Main Goal or Problem	Expected System Interaction	Requirement Source / Evidence
Retail Consumer/Buyer	Avoid buying spoiled seafood and maintain a personal record of freshness scans at local market stalls.	Signs in via Google OAuth 2.0, captures or uploads fish images, views classification results, reviews personal scan history, deletes past history entries, and manages profile details.	Buyer reports regarding uncertainty during seafood purchases and desire for persistent scan history.
Unauthenticated Guest	Test the freshness classification tool quickly without creating an account or signing in.	Captures or selects a single fish image, submits it for classification, and views the Fresh or Not Fresh result and confidence score on screen. Scan is not saved to history.	Initial user feedback requesting immediate application trial without upfront authentication friction.
System Admin	Maintain database integrity, monitor DRF API endpoint health, and manage system deployment logs.	Manages Django Admin console, inspects PostgreSQL database records, and monitors authentication logs.	System administration requirements and least-privilege security controls.

6. Part B - Functional and Non-Functional Requirements

Write requirements so another team member can implement and test them without guessing. Give each requirement a unique ID. Functional requirements describe system behavior or services. Non-functional requirements describe measurable quality expectations or constraints. Use actual users, data, rules, and project conditions rather than generic statements.

Requirement ID / Type	Requirement Statement	Source / Rationale	Priority	Verification / Acceptance Evidence
FR-001	The system shall authenticate users via Google Sign-In, obtaining a Google ID token in Flutter and verifying it on the Django REST Framework backend (POST /api/v1/auth/google/) to grant authorized access to protected endpoints.	Secure identity management via Google OAuth 2.0 / OpenID Connect without maintaining custom password tables.	Must	Sending a valid Google ID token returns HTTP 200 with user profile claims; invalid tokens return HTTP 401 Unauthorized.
FR-002	The system shall allow both unauthenticated Guests and authenticated users (Consumers/Vendors) to select a fish image using either a single still-image camera preview capture or a file upload from local	Core image input mechanism for visual freshness evaluation.	Must	Flutter UI allows capturing a single photo or selecting a file from gallery, placing image bytes into state.

	storage.			
FR-003	The system shall enforce image validation (JPEG or PNG format; file size <= 5MB) on both the Flutter client before transmission and the Django REST Framework backend upon request reception.	Prevents backend model errors and avoids bandwidth overload.	Must	Selecting a non-image file or an image > 5MB displays a client alert and causes Django to reject the payload with HTTP 400 Bad Request.
FR-004	The internal Django application tier shall decode, resize, normalize, and pass submitted fish images to the internal MobileNetV3 model to return a JSON response containing label (Fresh / Not Fresh), confidence score, and scan_id (if saved).	Primary system automated freshness classification functionality.	Must	POST /api/v1/classify/ returns JSON formatted as {"label": "Fresh", "confidence": 0.92, "scan_id": 101}.
FR-005	The system shall save successful classification records to PostgreSQL via Django ORM only when requested by an authenticated Consumer or Vendor; Guest scans shall not be saved.	User history tracking and data privacy boundaries.	Must	Authenticated requests populate classification_history in PostgreSQL with a valid user_id; Guest requests return scan_id: null and leave database tables unchanged.
FR-006	The system shall allow authenticated users to view their chronological scan list (GET /api/v1/history/), view record details (GET /api/v1/history/{id}/), and permanently delete specific records (DELETE /api/v1/history/{id}/).	User history control and record management.	Must	Dio client renders history list from API; sending DELETE removes the record from PostgreSQL and returns HTTP 204 No Content.
FR-007	The system shall display account profile details (GET /api/v1/profile/) and allow updating display name preferences (PATCH /api/v1/profile/) for authenticated users.	Profile management functionality.	Must	Profile UI renders authenticated user's Google name/email and updates fields in PostgreSQL upon valid submission.
FR-008	The system shall display a mandatory decision-support disclaimer on classification output screens stating that results provide decision support only and do not replace laboratory food safety testing.	Scope restriction and user risk mitigation.	Must	Static disclaimer text block is permanently visible on both Flutter Web and Mobile classification result views.
NFR-001	The end-to-end classification request (POST /api/v1/classify/) shall complete inference and return a response within < 3seconds under standard mobile network conditions.	System performance and usability target.	Must	Network latency logs measuring execution time from Flutter request submission to JSON rendering.

NFR-002	The internal MobileNetV3 machine learning model shall achieve a target freshness classification accuracy rate $\geq 85\%$ on validation datasets.	Quality outcome indicator.	Must	Model evaluation matrix showing top-1 accuracy, precision, and recall metrics on test samples.
NFR-003	The Flutter client shall never establish direct database connections to PostgreSQL; all data access must pass through Django REST Framework endpoints secured by HTTP Bearer tokens.	Architecture security constraint.	Must	Code inspection verifying database credentials reside exclusively within Django server settings.
NFR-004	The system shall gracefully handle network errors during classification or history operations by displaying a clear retry prompt, explicitly notifying the user that active internet connectivity is required.	Robustness and usability requirement.	Must	Disconnecting network during submission or history retrieval triggers a Flutter snackbar/dialog error alert without app crash.

Requirement-writing rule

Prefer a testable form such as “The system shall ...”. Keep one requirement per statement. Replace vague words such as fast, secure, easy, user-friendly, reliable, or real-time with measurable conditions whenever the project can define them. A functional requirement should identify the actor or trigger, required behavior, important data or rule, and expected outcome. A non-functional requirement should state the quality attribute, condition, target or threshold, and how it can be verified.

7. Part C - User Roles and Permissions

Define the actual access level of every system role. The role-permission matrix must agree with the use cases/user stories and with the authentication and authorization controls shown in the architecture. Do not give every user full access simply to simplify implementation.

User Role	Purpose / Main Tasks	Allowed Actions	Restricted /	Access	Authentication / Permission Rule
Guest / Unauthenticated User	Test system classification functionality without registering.	Capture still image or pick gallery file; submit image for classification; view on-screen result, confidence, and decision-support disclaimer.	Cannot save scan to history; cannot view history or profile screens; cannot access protected API endpoints.	None (No backend data persistence).	Unauthenticated (AllowAny).
Consumer / Retail Buyer	Evaluate seafood freshness during market purchases and track	All Guest actions; sign in via Google OAuth 2.0;	Cannot view, edit, or delete scan histories or profile data	Own user account record and personal classification history	Authenticated via Google OAuth ID token

	personal scan history.	automatically save scans to personal history; view history list and detail views; delete personal history records; view and update profile.	belonging to other users; cannot access admin backend.	records in PostgreSQL.	(IsAuthenticated), filtered by Django ORM user_id.
Fish Vendor / Seller	Inspect market stall inventory freshness and present visual classification proof to buyers.	All Guest actions; sign in via Google OAuth 2.0; automatically save scans to personal history; view history list and detail views; delete personal history records; view and update profile.	Cannot view, edit, or delete scan histories or profile data belonging to other users; cannot access admin backend.	Own user account record and personal classification history records in PostgreSQL.	Authenticated via Google OAuth ID token (IsAuthenticated), filtered by Django ORM user_id.

Role and access review checks:

- Assign only the permissions required for each role to complete its responsibilities.
- Identify which records a user may view, create, edit, approve, delete, export, or administer.
- Separate sensitive or administrative actions from ordinary user actions and define where authorization must be enforced.
- Ensure each permission is reflected in at least one requirement, use case/user story, or explicit out-of-scope rule.

8. Part D - System Scope, Boundaries, Assumptions, and Constraints

Selected methodology from Activity 4: Scrum (Adaptive/Agile, 2-week iterative sprints).

Prescribed architecture :Flutter Web/Mobile (Riverpod, Dio) -> Django REST Framework (Internal MobileNetV3 ML) -> PostgreSQL (Django ORM).

System objective / intended outcome (2-3 sentences):

Provide an automated image-based freshness classification tool delivering a decision-support label (Fresh / Not Fresh) with confidence percentage in under 3 seconds per scan, saving scan histories for authenticated Consumers and Vendors while protecting user privacy.

Scope and Boundary Worksheet

For each major module or feature area, state what the semester project will deliver, what will remain outside the project boundary, who uses it, what data it needs, and which dependencies or constraints affect implementation.

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
1. User Authentication	Google Sign-In via Google OAuth 2.0 / OpenID Connect; Google ID token verification in DRF.	Custom email/password registration, password resets, SMS/MFA.	Consumer, Vendor	Google ID token, email, display name, user ID.	Google OAuth 2.0 API.	Google OAuth is the only external service allowed in architecture.
2. Image Capture &	Single still-image camera capture	Continuous video stream inference,	Guest, Consumer,	Raw image file/bytes, image	Flutter image_picker / camera plugin.	Single image capture

Upload	via camera preview; gallery file selection.	multi-fish bounding box detection.	Vendor	dimensions, file type.		maintains response speeds < 3 seconds.
3. ML Classification	Django internal preprocessing (resizing, normalization) and MobileNetV3 model inference.	External ML API calls, hardware gas/chemical sensor inputs.	Guest, Consumer, Vendor	Processed tensor image, output label, confidence score.	Internal MobileNetV3 model in Django application tier.	ML model resides inside Django application tier; not an external API.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Module / Area	In Scope	Out of Scope	User(s)	Main Data	Dependency / External Service	Constraint / Rationale
4. Scan History	Auto-saving authenticated scans to PostgreSQL; viewing history list online; retrieving single detail; deleting records.	Saving Guest scans; retroactive history saving after late login; offline history caching; PDF export.	Consumer, Vendor	scan_id, image_path, label, confidence, created_at.	Django ORM, Render PostgreSQL.	History retrieval requires active connectivity; Guest results are non-persistent by design.
5. Profile Management	Viewing account details (GET) and updating display name (PATCH).	Role management, enterprise account settings, billing.	Consumer, Vendor	User ID, email, display name, account creation date.	Django REST Framework profile endpoints.	Profile access strictly scoped to the authenticated session owner.
6. System Disclaimer	Permanent display of decision-support disclaimer on result displays.	Automated food safety certification, legal compliance export.	Guest, Consumer, Vendor	Static disclaimer text block.	None.	Mitigates liability by explicitly stating tool is decision support only.

Boundary rule

“In scope” means the team commits to analyze, implement, test, and demonstrate the item within the semester. “Out of scope” means it is intentionally excluded from the current version, even if it may be useful in the future. Assumptions are conditions believed to be true; dependencies are external items the system relies on; constraints are limits the team must work within. All four should be explicit to prevent uncontrolled scope growth.

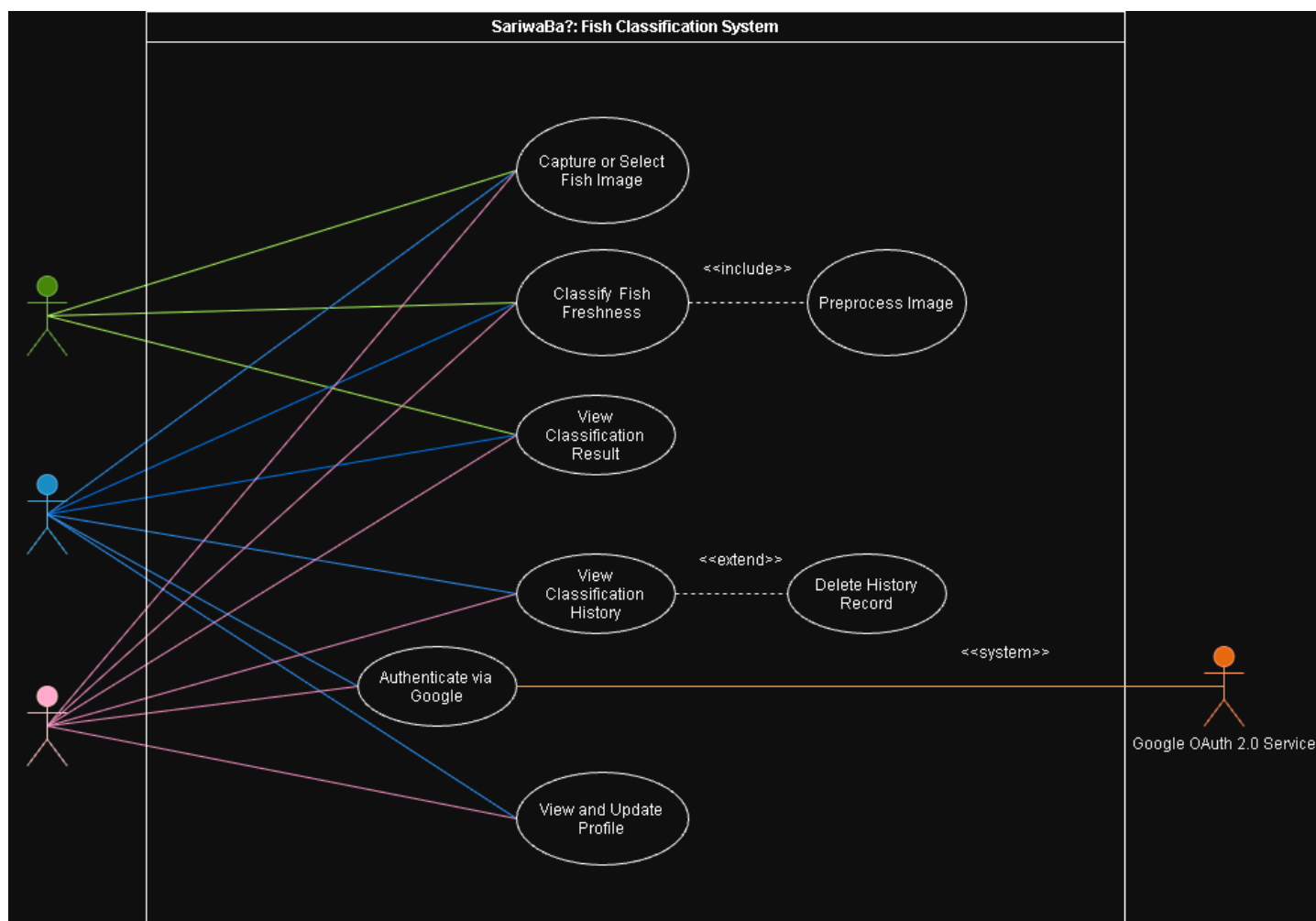
9. Part E - System Use Case Diagram and Use-Case/User-Story Refinement

Model the required behavior from the user's point of view. Every group must first create one complete UML use case diagram for its own approved semester system. The diagram must be based on the actual user roles, functional requirements, scope, and permissions defined in this activity. After the diagram is completed, refine the major use cases and/or user stories in the worksheet below. Teams using a predictive approach may emphasize detailed use-case descriptions, while teams using an adaptive/Agile approach may also maintain user stories and acceptance criteria. The diagram and the written requirements must remain consistent.

Mandatory Use Case Diagram Requirement

Each group must create a UML use case diagram for its own approved system. The diagram is a required Activity 5 output and must be inserted in the SRS or attached as a clearly labeled figure.

- Draw one system boundary rectangle labeled with the official system title.
- Place all major actors or user roles outside the system boundary.
- Show the major user-goal use cases inside the boundary using clear verb-based names (for example: Submit Report, Approve Request, View Map, or Manage Account).
- Connect actors to the use cases they are allowed to perform. Use <<include>> and <<extend>> only when they are logically justified.
- Ensure all Must/High-priority functional features are represented and that the diagram agrees with the role-permission matrix, system scope, and functional requirements.
- Do not place Flutter, the REST API,s PostgreSQL, database tables, screens, or other internal implementation components as actors or use cases. These belong to architecture/design artifacts, not the UML use case diagram.
- Required files: readable PNG of the final use case diagram plus the editable source file (for example .drawio, .fig, or equivalent).



ID	Actor / User	Goal / Need	Trigger / Input	Main Interaction or User Story	Expected Result / Acceptance
UC--01	Consumer, Vendor	Authenticate into the system using Google.	User taps "Sign in with Google" button.	As a Consumer/Vendor, I want to sign in with Google so that I can securely save and manage my scan history.	Flutter receives Google ID token, sends it to POST /api/v1/auth/google/, and receives verified session claims.
UC-02	Consumer, Vendor /	Submit a fish image for freshness classification.	User selects camera capture or gallery picker.	As a user, I want to capture or upload a fish photo so that the system can analyze its visual freshness.	Selected image is validated for type/size and rendered in the upload preview container.
UC-03	Guest, Consumer, Vendor	Obtain freshness classification output.	User taps "Classify Freshness" button.	As a user, I want to receive an immediate freshness label (Fresh/Not Fresh) and confidence score so that I can evaluate the fish quality.	Client receives JSON payload from /api/v1/classify/ in < 3 seconds and displays result modal with disclaimer.
UC-04	Consumer, Vendor	Save classification record to history.	Successful completion of UC-03 by authenticated user.	As an authenticated user, I want my scan automatically saved so that I can review it in my history later.	Flutter sends GET /api/v1/history/ with Bearer token; UI renders chronological list of images, labels, and timestamps.
UC-05	Consumer, Vendor	Review previous scan history.	User navigates to the "History" screen.	As an authenticated user, I want to view my past scan records so that I can track previous fish evaluations.	Client fetches GET /api/v1/history/ and renders list of past scans sorted chronologically.
UC-06	Consumer, Vendor	Delete a specific scan history record.	User taps "Delete" icon on a history detail item.	As an authenticated user, I want to delete a history entry so that I can remove unwanted records.	Client dispatches DELETE /api/v1/history/{id}/; Django ORM removes record from PostgreSQL and returns HTTP 204.
UC-07	Consumer, Vendor	View and update profile settings.	User navigates to "Profile" screen.	As an authenticated user, I want to view and edit my profile details so that my account remains up to date.	Client fetches GET /api/v1/profile/ and dispatches PATCH /api/v1/profile/ to update profile attributes.

Refinement rule

For a use case, identify the actor, goal, precondition when relevant, main flow, alternative/error flow, and expected postcondition. For a user story, use a form such as "As a [role], I want [capability], so that [value]" and attach acceptance criteria. Every Must/High priority functional requirement should link to at least one use case, user story, or clearly defined system event.

10. Part F - Feature Priorities, Acceptance Criteria, and Project Constraints

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Feature prioritization	Priority scheme (MoSCoW) for semester deliverables.	Must Have: Google OAuth 2.0 Auth (FR-001), Image Capture/Upload (FR-002), Client/Server Validation (FR-003), Internal ML Classification (FR-004), History Persistence & Management (FR-005, FR-006), Profile Management (FR-007), Disclaimer Display (FR-008) Won't Have: Custom password auth, Guest history saving, continuous video stream scanning, offline history caching
Acceptance criteria	Observable conditions for feature completion.	1. POST /api/v1/classify/ completes within <3 seconds over network. 2. Guest scans return scan_id: null and create zero PostgreSQL rows. 3. Authenticated scans populate classification_history table correctly. 4. Non-image files or files >5 MB return HTTP 400 error alerts
Business rules	Domain logic and validation constraints.	1. Classification output label is strictly binary (Fresh or Not Fresh). 2. Guest classification results are non-persistent and lost upon page/screen navigation. 3. Logging in after a Guest scan does not retroactively save that scan. 4. Decision-support disclaimer must accompany every classification result display. 5. Decision support disclaimer must accompany classification displays.
Project constraints	Technical, architecture, and timeline limits.	1.Tech Stack: Flutter (Dart, Riverpod, Dio) → Django REST Framework (Python 3.x) → PostgreSQL.

		2. Deployment: Vercel (Flutter Web), Render Web Service (Django REST Framework + ML), Render PostgreSQL. 3. Architecture Constraint: Flutter must never connect directly to PostgreSQL.
Data and privacy constraints	Sensitive data and access protection.	User email and Google ID stored securely in PostgreSQL. Scan history records isolated by user ID via Django ORM IsAuthenticated permissions.
External-service dependencies	External services and fallback handling.	Google OAuth 2.0 / OpenID Connect is the sole external service. Internal MobileNetV3 ML model runs inside Django, eliminating third-party ML API dependencies.
Performance / reliability targets	Latency and model accuracy targets.	End-to-end response time < 3 seconds per scan; internal MobileNetV3 accuracy target > 85% on test dataset.
Validation and error handling	Error response behavior.	API returns HTTP 400 for invalid image uploads, HTTP 401 for missing/invalid Bearer tokens, HTTP 404 for missing history records, and HTTP 500 for backend model execution faults with user-friendly Flutter error alerts.

Recommended minimum requirement evidence in the project repository or shared workspace:

docs/requirements/ system-requirements-specification.docx or .pdf requirements-to-feature-matrix.xlsx or .csv use-cases-or-user-stories/ role-permission-matrix/ scope-and-constraints/ review-notes/ README.md - current requirements baseline/version and status

11. Part G - Construct the System Requirements Specification

Assemble the completed analysis into one coherent SRS. The document must be project-specific and internally consistent. Do not copy generic requirements from another system. Use stable requirement IDs so changes can be traced during implementation and testing.

No.	Required SRS Element	What Must Be Shown
-----	----------------------	--------------------

1	Document control and project identity	Official system title, group, version/date, selected methodology, and prescribed Flutter Web REST API-PostgreSQL architecture.
2	Problem and target users	Community/organizational problem, current situation, intended beneficiaries, primary users, and stakeholders.
3	System objectives	Specific outcomes the system is intended to achieve without describing unsupported future features.
4	Scope and boundaries	In-scope functions/modules, explicit out-of-scope items, assumptions, dependencies, and constraints.
5	User roles and permissions	Role definitions, allowed/restricted actions, data-access scope, and authentication/authorization needs.
6	Functional requirements	Uniquely identified, clear, atomic, feasible, and testable statements of required system behavior.
7	Non-functional requirements	Measurable quality requirements for relevant areas such as security, performance, usability, reliability, compatibility, maintainability, and deployment.
8	Use cases and/or user stories	Actors/roles, user goals, main interactions or stories, alternative/error handling when relevant, and expected results.
9	Features and priorities	Feature list grouped from requirements with a consistent priority scheme and semester delivery decision.
10	Acceptance criteria and business rules	Observable completion conditions, validations, calculations, approvals, transitions, and other domain rules.
11	Architecture and data alignment	Which requirements are handled by Flutter Web, the REST API/business logic, PostgreSQL, and any approved external services.
12	Traceability and review status	Requirement IDs linked to features/use cases or stories, architecture/data components, acceptance evidence, open issues, and review status.

Minimum SRS Rule

Every Must/High-priority feature must be supported by one or more functional requirements and any relevant non-functional requirements. Each requirement must have a unique ID, a clear source or rationale, a priority, and a verification/acceptance method. Requirements must remain feasible within the selected methodology, semester scope, and prescribed Flutter Web -> REST API -> PostgreSQL architecture.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the analysis and traceability logic expected in Activity 5. Replace every generic label with the actual community problem, users, requirements, features, roles, constraints, architecture components, and acceptance evidence of your own semester project.

Reference Requirements Analysis and Traceability Flow

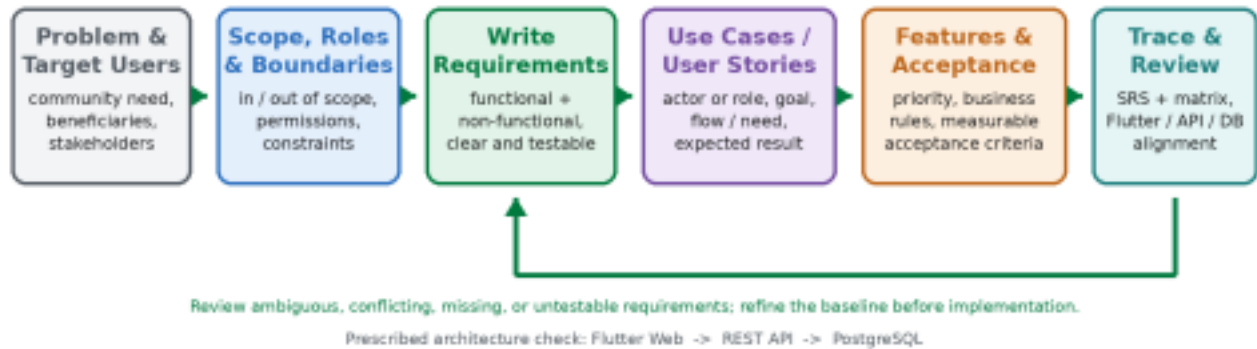


Figure 1. Reference requirements analysis and traceability flow.

Step	Reference Flow
1	Start from the verified community/organizational problem, target users, stakeholders, and requirement evidence.
2	Define system scope, role permissions, boundaries, assumptions, dependencies, and project constraints.
3	Write uniquely identified functional and non-functional requirements that are clear, feasible, and testable.
4	Refine the behavior through use cases and/or user stories that reflect the team's selected development methodology.
5	Group requirements into prioritized features and define measurable acceptance criteria and business rules.
6	Trace requirements to features, Flutter Web/API/PostgreSQL responsibilities, acceptance evidence, and review status; revise gaps before implementation.

13. Part H - Requirements-to-Feature Traceability Matrix

System module / feature set: **Core System Baseline (Authentication, Classification, History, Profile)**

Requirements baseline version / date: **Version 1.1 (Draft Baseline — October 4, 2026)**

Requirement ID	Requirement Summary	Linked Feature & Use Case	Stack Component / Data	Acceptance / Verification & Status
FR-001	Google OAuth authentication	Google Sign-In (UC-01).	Flutter Google Sign-In → DRF /api/v1/auth/google/ → PostgreSQL users table.	Draft: Valid ID token returns user session claims; invalid token

				returns HTTP 401.
FR-002	Camera capture / file pick.	Image Selection (UC-02).	Flutter image_picker plugin → Dio FormData payload.	Draft: Captures single still image or selects gallery file into state.
FR-003	Format & size validation.	Upload Validation (UC-02).	Flutter client validation logic & Django REST Framework serializers.	Draft: Files non-JPEG/PNG or > 5MB rejected with HTTP 400 alert.
FR-004	Internal MobileNetV3 ML inference.	Freshness Classification (UC-03).	Django Application Tier (Internal MobileNetV3 Model Component).	Draft: /api/v1/classify/ returns JSON with label, confidence, scan_id.
FR-005	Scan history persistence rule.	History Logging (UC-04).	Django ORM → PostgreSQL classification_history table.	Draft: Saves record only if authenticated; Guest scans return scan_id: null.
FR-006	View & delete scan history.	History Management (UC-05, UC-06).	Flutter History Screen → DRF /api/v1/history/ → Django ORM.	Draft: GET lists user scans; DELETE /api/v1/history/{id}/ removes row.
FR-007	View & update user profile.	Profile Management (UC-07).	Flutter Profile Screen → DRF /api/v1/profile/ → PostgreSQL users.	Draft: GET displays profile; PATCH updates display name in database.
FR-008	Decision-support disclaimer.	Disclaimer Display (UC-03).	Flutter UI static component / Result modal view.	Draft: Disclaimer text permanently rendered under output label.
NFR-001	Classification speed < 3 seconds..	System Performance Baseline.	Dio client network timer & Django view execution profiler.	Draft: Total request duration logged under 3.0 seconds.
NFR-002	Target accuracy > 85%.	ML Quality Target.	MobileNetV3 validation evaluation matrix notebook.	Draft: Test evaluation matrix shows overall classification accuracy > 85%.
NFR-003	No direct database connection.	Security Architecture Boundary.	Flutter repository inspection & architectural code review.	Draft: Zero direct database drivers in Flutter codebase.
NFR-004	Network error handling.	Client Usability Baseline.	Flutter Dio error interceptor & UI alert dialogue.	Draft: Network disconnection triggers retry prompt without app crash.

14. Part I - Requirements Review and Baseline Approval

Before submission, conduct a structured group review and prepare the SRS for the online requirements review. Record issues that must be revised instead of silently changing the baseline. Confirm all of the following:

- Every requirement is necessary, understandable, uniquely identified, feasible within the semester, and testable or verifiable.
- Functional requirements, non-functional requirements, roles/permissions, scope, use cases/user stories, features, priorities, and acceptance criteria agree with one another.
- No Must/High-priority feature lacks supporting requirements or acceptance evidence, and no requirement is orphaned

from a feature, user need, or system-quality objective.

- The requirements are consistent with the selected development methodology and with the prescribed Flutter Web REST API-PostgreSQL architecture.
- Ambiguous, conflicting, duplicate, missing, or out-of-scope requirements are identified and assigned a revision decision before implementation begins.

Online review status: ☐ Ready for review ☒ Requires revision ☐ Reviewed with comments Reviewer / consultation source: Jean Y. Villasin, MIT

Date: October 4, 2026

Baseline version: Version 1.1 (Draft Proposal)

Most important revision identified during review: Pending official instructor review. Draft revision applied: Explicitly restricted scan history saving (FR-005) to authenticated Consumers/Vendors while enforcing non-persistence for Guest scans.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The community/organizational problem and target users are clearly stated and supported by project evidence.	☐ Every requirement has a unique ID and uses project-specific, understandable wording.
☐ Functional requirements describe required behavior rather than vague features or implementation details.	☐ Non-functional requirements are measurable or verifiable where practical.
☐ User roles, permissions, restricted actions, and data-access boundaries are consistent across the SRS.	☐ In-scope and out-of-scope items are explicit; assumptions, dependencies, and constraints are documented.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 10 of 12

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Checklist A	Checklist B
☐ Use cases and/or user stories are linked to actual roles and required system behavior.	☐ Features are prioritized consistently and acceptance criteria are observable and testable.
☐ Business rules, validation, error handling, and external-service dependencies are documented when relevant.	☐ Requirements are aligned with Flutter Web, the REST API, PostgreSQL, and approved external services.
☐ The requirements-to-feature matrix contains no unexplained orphan requirements or unsupported Must/High features.	☐ The SRS and traceability matrix agree with the selected development methodology and semester plan.
☐ The requirements baseline has been reviewed for ambiguity, conflict, duplication, feasibility, and scope creep.	☐ Every group member can explain assigned requirements, role permissions, acceptance criteria, and traceability decisions during consultation or review.

16. Submission Requirements

- Final System Requirements Specification in editable DOCX and PDF format, using the team's current requirements baseline/version.
- Completed Requirements-to-Feature Traceability Matrix in the SRS or as an accompanying spreadsheet/table file.
- Completed role-permission matrix, scope/boundary worksheet, use-case diagram and/or user-story set, feature-priority list, and acceptance-criteria evidence.
- Updated project GitHub repository or shared workspace containing the requirement artifacts under a folder such as docs/requirements/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA5_SystemName.
- Complete the online requirements review. If the instructor returns comments or identifies requirement defects, submit

the revised baseline and preserve a brief record of the changes made.

Academic Integrity and Competency Validation

The SRS must represent the team's own approved project, stakeholder evidence, scope decisions, and technical constraints. AI tools may be used to clarify terminology, refine wording, or check consistency, but generated requirements must not be accepted without verification against the team's actual system. Every member is responsible for understanding the requirements assigned to the group. The instructor may ask any member to explain the source, purpose, priority, permission rule, acceptance criterion, or implementation layer of a requirement without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, ambiguous, inconsistent, untestable, outside the approved scope, or not traceable to the team's actual project. Requirements returned during the online review should be revised before they are used as an implementation baseline.

Criterion	Evidence for Full Credit	Points
Problem, target users, and stakeholder analysis	The SRS clearly identifies the real community/organizational problem, current situation, target users, stakeholders, goals, and evidence used to derive requirements.	8
Functional requirements	Functional requirements are complete for the approved scope, uniquely identified, project specific, atomic, feasible, clear, prioritized, and testable.	14
Non-functional requirements	Relevant quality requirements are measurable/verifiable and appropriately address security, performance, usability, reliability, compatibility, maintainability, deployment, or other project needs.	10

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 11 of 12

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 5

Criterion	Evidence for Full Credit	Points
User roles, permissions, and scope boundaries	Roles, allowed/restricted actions, data access, in-scope/out-of-scope items, assumptions, dependencies, and constraints are accurate and mutually consistent.	10
Use cases/user stories and feature priorities	Use cases and/or user stories correctly represent user goals and required behavior; features are logically grouped, linked to requirements, and prioritized for the semester.	10
Acceptance criteria, business rules, and constraints	Acceptance conditions are observable/testable; validation, calculations, approvals, errors, external dependencies, and important project constraints are documented when relevant.	8
Requirements-to-feature traceability and architecture alignment	The matrix links requirements to features/use cases or stories, Flutter Web/API/PostgreSQL responsibilities or data, and acceptance evidence without unexplained gaps or contradictions.	12
SRS quality and online requirements review	The document is organized, readable, versioned, internally consistent, submitted correctly, and reviewed for ambiguity, duplication, conflict, feasibility, traceability, and scope before implementation.	8

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which requirement was the most difficult to make clear and testable, and what did your group change?

Answer: FR-005 (Scan History Persistence Rule). The initial draft vaguely implied all scans were saved automatically. We changed it to explicitly decouple persistence by role: Guest scans are strictly ephemeral (return scan_id: null with zero PostgreSQL writes), whereas authenticated Consumer/Vendor scans automatically persist to PostgreSQL via Django ORM, making acceptance testing clear and verifiable.

Which requested feature or capability was placed out of scope for this semester, and why?

Answer: Continuous video stream ML inference and offline ML classification/history caching. Video streaming was excluded because real-time frame transmission over mobile networks breaches our < 3 seconds latency target. Offline ML/caching was excluded to avoid bloating the Flutter app bundle and to maintain our architecture where MobileNetV3 inference runs inside Django over active network connections.

Which user role has the most security-sensitive permissions, and what control should protect those actions?

Answer: Authenticated Consumers and Vendors, as they execute write, read, and delete operations on persistent user profile and scan history data (FR-006, FR-007). These actions are protected by Google OAuth 2.0 ID token authentication (IsAuthenticated) and Django ORM user-level data isolation (filter(user=request.user)), preventing unauthorized cross-user access or IDOR vulnerabilities.

Which non-functional requirement has the greatest effect on your Flutter Web-REST API-PostgreSQL architecture?

Answer: NFR-003 (No Direct Database Connections). It forces complete structural separation between the Flutter client and PostgreSQL database. Flutter relies entirely on Riverpod state management and Dio HTTP requests to communicate with Django REST Framework, which securely runs internal MobileNetV3 inference and performs PostgreSQL transactions via Django ORM.

What requirement changed or became clearer after the requirements review, and how will your selected methodology manage that change?

Answer: The Google ID token verification flow (FR-001) and Guest scan non-persistence (FR-005). Our Scrum methodology manages these refined details through iterative backlog refinement, converting the clarified requirements into user stories with testable acceptance criteria verified during two-week sprint reviews.

